Vel Tech Group of Institutions

Name: GOPICHAND KAKIVAI
Email: vtu29748@veltech.edu.in
Roll no: vtu29748
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS1L3

VTU_DAA_Week 3_PAH

Attempt : 1
Total Mark : 50
Marks Obtained : 48.5

Section 1 : Coding

1. Problem Statement:

Javier, a landscape architect, is designing a new botanical garden and needs to determine the outer boundary that will enclose all the selected plant species.

He has gathered coordinates of key points where the plants are located and needs to calculate the convex hull to create a protective garden boundary. To determine this boundary, Javier decides to use Graham's Scan algorithm.

Help Javier compute the convex hull by implementing the algorithm to design the garden's boundary.

Input Format

The first line of input consists of an integer n, representing the number of plant species locations.

The next n lines contain two integers x and y, representing the coordinates of each location.

Output Format

The output prints "Convex Hull:" followed by the coordinates of the points forming the convex hull in counterclockwise order, with each point on a new line.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
0 0
0 1
1 0
1 1

Output: Convex Hull:
(0, 0)
(1, 0)
(1, 1)
(0, 1)

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

typedef struct {
    int x, y;
} Point;

Point pivot;

int distSq(Point p1, Point p2) {
    return (p1.x - p2.x) * (p1.x - p2.x) + (p1.y - p2.y) * (p1.y - p2.y);
}
```

```
int orientation(Point p, Point q, Point r) {
    int val = (q.y - p.y) * (r.x - q.x) - (q.x - p.x) * (r.y - q.y);
    if (val == 0) return 0;
    return (val > 0) ? 1 : 2;
}

int compare(const void* vp1, const void* vp2) {
    Point* p1 = (Point*)vp1;
    Point* p2 = (Point*)vp2;
    int o = orientation(pivot, *p1, *p2);
    if (o == 0)
        return (distSq(pivot, *p2) >= distSq(pivot, *p1)) ? -1 : 1;
    return (o == 2) ? -1 : 1;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    if (n < 3) {
        Point p[n];
        for(int i = 0; i < n; i++) scanf("%d %d", &p[i].x, &p[i].y);
        printf("Convex Hull:\n");
        for(int i = 0; i < n; i++) printf("(%d, %d)\n", p[i].x, p[i].y);
        return 0;
    }

    Point pts[n];
    for (int i = 0; i < n; i++) {
        scanf("%d %d", &pts[i].x, &pts[i].y);
    }

    int ymin = 0;
    for (int i = 1; i < n; i++) {
        if (pts[i].y < pts[ymin].y || (pts[i].y == pts[ymin].y && pts[i].x < pts[ymin].x))
            ymin = i;
    }

    Point temp = pts[0];
    pts[0] = pts[ymin];
    pts[ymin] = temp;

    pivot = pts[0];
```

```
    qsort(&pts[1], n - 1, sizeof(Point), compare);

    int m = 1;
    for (int i = 1; i < n; i++) {
        while (i < n - 1 && orientation(pivot, pts[i], pts[i + 1]) == 0) i++;
        pts[m++] = pts[i];
    }

    if (m < 3) {
        printf("Convex Hull:\n");
        for(int i = 0; i < m; i++) printf("(%d, %d)\n", pts[i].x, pts[i].y);
        return 0;
    }

    Point stack[n];
    int top = 0;
    stack[top++] = pts[0];
    stack[top++] = pts[1];
    stack[top++] = pts[2];

    for (int i = 3; i < m; i++) {
        while (top > 1 && orientation(stack[top - 2], stack[top - 1], pts[i]) != 2) top--;
        stack[top++] = pts[i];
    }

    printf("Convex Hull:\n");
    for (int i = 0; i < top; i++) {
        printf("(%d, %d)\n", stack[i].x, stack[i].y);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement:

Eva, a botanist, is surveying a nature reserve to protect rare plant species. During her survey, she marked a special tree, known as the "guardian tree," and now needs to map out the perimeter of the reserve, ensuring the

guardian tree is included in the boundary. To do this, Eva needs to compute the convex hull of all the points, including the coordinates of the guardian tree.

Your task is to compute the convex hull, ensuring that the special guardian tree is included in the boundary.

Example

Input:

5

0.0 0.0

2.0 0.0

2.0 2.0

0.0 2.0

1.0 1.0

3.0 1.0

Output:

0 0

2 0

3 1

2 2

0 2

Explanation:

Input Points: (0.0, 0.0), (2.0, 0.0), (2.0, 2.0), (0.0, 2.0), (1.0, 1.0), (3.0, 1.0).

Pivot Selection: The point with the lowest y-coordinate (and leftmost in case of a tie) is selected as the pivot: (0, 0).

Sorting by Polar Angle: Points are sorted by the polar angle relative to the pivot: (0, 0), (2, 0), (3, 1), (2, 2), (0, 2).

Convex Hull Construction: Using counterclockwise turns, the points forming the convex hull are: (0, 0), (2, 0), (3, 1), (2, 2), (0, 2).

Input Format

The first line contains an integer N, representing the number of trees originally surveyed.

The next N lines each contain two double values x and y, representing the coordinates of the surveyed trees.

The last line contains two double values x and y, representing the coordinates of the guardian tree.

Output Format

The output prints the coordinates of the points forming the convex hull in counterclockwise order, ensuring the special guardian tree is included. Each coordinate should be printed on a new line, with the values of x and y separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
0.0 0.0
2.0 0.0
2.0 2.0
0.0 2.0
1.0 1.0
3.0 1.0

Output: 0 0
2 0
3 1
2 2
0 2

Answer

```
#include <stdio.h>
#include <stdlib.h>
```

```
#include <math.h>

typedef struct {
    double x, y;
} Point;

Point pivot;

double distSq(Point p1, Point p2) {
    return (p1.x - p2.x) * (p1.x - p2.x) + (p1.y - p2.y) * (p1.y - p2.y);
}

int orientation(Point p, Point q, Point r) {
    double val = (q.y - p.y) * (r.x - q.x) - (q.x - p.x) * (r.y - q.y);
    if (fabs(val) < 1e-9) return 0;
    return (val > 0) ? 1 : 2;
}

int compare(const void* vp1, const void* vp2) {
    Point* p1 = (Point*)vp1;
    Point* p2 = (Point*)vp2;
    int o = orientation(pivot, *p1, *p2);
    if (o == 0)
        return (distSq(pivot, *p2) >= distSq(pivot, *p1)) ? -1 : 1;
    return (o == 2) ? -1 : 1;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int totalPoints = n + 1;
    Point pts[totalPoints];

    for (int i = 0; i < n; i++) {
        scanf("%lf %lf", &pts[i].x, &pts[i].y);
    }
    scanf("%lf %lf", &pts[n].x, &pts[n].y);

    int ymin = 0;
    for (int i = 1; i < totalPoints; i++) {
        if (pts[i].y < pts[ymin].y || (fabs(pts[i].y - pts[ymin].y) < 1e-9 && pts[i].x <
pts[ymin].x))
```

```
            ymin = i;
        }

        Point temp = pts[0];
        pts[0] = pts[ymin];
        pts[ymin] = temp;

        pivot = pts[0];
        qsort(&pts[1], totalPoints - 1, sizeof(Point), compare);

        int m = 1;
        for (int i = 1; i < totalPoints; i++) {
            while (i < totalPoints - 1 && orientation(pivot, pts[i], pts[i + 1]) == 0) i++;
            pts[m++] = pts[i];
        }

        if (m < 3) {
            for (int i = 0; i < m; i++) {
                printf("%g %g\n", pts[i].x, pts[i].y);
            }
            return 0;
        }

        Point stack[totalPoints];
        int top = 0;
        stack[top++] = pts[0];
        stack[top++] = pts[1];
        stack[top++] = pts[2];

        for (int i = 3; i < m; i++) {
            while (top > 1 && orientation(stack[top - 2], stack[top - 1], pts[i]) != 2) top--;
            stack[top++] = pts[i];
        }

        for (int i = 0; i < top; i++) {
            printf("%g %g\n", stack[i].x, stack[i].y);
        }

        return 0;
    }
```

Status : Correct **Marks : 10/10**

3. Problem Statement

Nathan is learning about computational geometry and has come across the concept of a Convex Hull.

A Convex Hull is a convex polygon that encompasses a set of points in a two-dimensional plane, such that no point lies within the polygon's interior.

Nathan is particularly interested in determining whether a given point lies inside or outside the convex hull formed by a set of points.

Help Nathan by writing a program that checks whether a given point lies inside or outside the convex hull formed by a set of points.

Example

N = 7,

Points: {(1, 1), (2, 1), (3, 1), (4, 1), (4, 2), (4, 3), (4, 4)},

Query: X = 3, Y = 2

Below is the image of the plotting of the given points:

The query (3, 2) lies inside the Convex Hull.

Input Format

The first line of input consists of an integer N, representing the number of points in the set.

The following N lines, each containing two integers x and y, represent the coordinates of the points in the set.

The last line consists of two integers px and py, representing the coordinates of the test point.

Output Format

The output prints whether the given test points lie inside or outside the Convex Hull.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 7
1 1
2 1
3 1
4 1
4 2
4 3
4 4
3 2
Output: The point (3, 2) lies inside the Convex Hull.

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

typedef struct {
    int x, y;
} Point;

Point pivot;

int distSq(Point p1, Point p2) {
    return (p1.x - p2.x) * (p1.x - p2.x) + (p1.y - p2.y) * (p1.y - p2.y);
}

int orientation(Point p, Point q, Point r) {
    int val = (q.y - p.y) * (r.x - q.x) - (q.x - p.x) * (r.y - q.y);
    if (val == 0) return 0;
    return (val > 0) ? 1 : 2;
}
```

```
int compare(const void* vp1, const void* vp2) {
    Point* p1 = (Point*)vp1;
    Point* p2 = (Point*)vp2;
    int o = orientation(pivot, *p1, *p2);
    if (o == 0)
        return (distSq(pivot, *p2) >= distSq(pivot, *p1)) ? -1 : 1;
    return (o == 2) ? -1 : 1;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    Point pts[n];
    for (int i = 0; i < n; i++) {
        scanf("%d %d", &pts[i].x, &pts[i].y);
    }

    Point query;
    scanf("%d %d", &query.x, &query.y);

    int ymin = 0;
    for (int i = 1; i < n; i++) {
        if (pts[i].y < pts[ymin].y || (pts[i].y == pts[ymin].y && pts[i].x < pts[ymin].x))
            ymin = i;
    }

    Point temp = pts[0];
    pts[0] = pts[ymin];
    pts[ymin] = temp;

    pivot = pts[0];
    qsort(&pts[1], n - 1, sizeof(Point), compare);

    int m = 1;
    for (int i = 1; i < n; i++) {
        while (i < n - 1 && orientation(pivot, pts[i], pts[i + 1]) == 0) i++;
        pts[m++] = pts[i];
    }

    if (m < 3) {
```

```
        printf("The point (%d, %d) lies outside the Convex Hull.\n", query.x, query.y);
        return 0;
    }

    Point hull[n];
    int top = 0;
    hull[top++] = pts[0];
    hull[top++] = pts[1];
    hull[top++] = pts[2];

    for (int i = 3; i < m; i++) {
        while (top > 1 && orientation(hull[top - 2], hull[top - 1], pts[i]) != 2) top--;
        hull[top++] = pts[i];
    }

    int inside = 1;
    for (int i = 0; i < top; i++) {
        int o = orientation(hull[i], hull[(i + 1) % top], query);
        if (o == 1) {
            inside = 0;
            break;
        }
    }

    if (inside) {
        printf("The point (%d, %d) lies inside the Convex Hull.\n", query.x, query.y);
    } else {
        printf("The point (%d, %d) lies outside the Convex Hull.\n", query.x, query.y);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Ragu, an aspiring computer scientist, is currently learning about the concept of finding the closest pair of points in a plane. He wants to test his skills by solving a programming challenge related to this concept.

Given a set of points in a 2D plane, Ragu needs to find the closest pair of points and calculate their Euclidean distance.

Input Format

The first line of input consists of an integer N, representing the number of points in the plane.

The following N lines, each consisting of two space-separated real numbers x and y, represent the coordinates of a point.

Output Format

The first line of output prints the coordinates of the two points forming the closest pair in the below format:

"Closest pair of points: (x1, y1) and (x2, y2)"

The second line prints "Distance: <distance>" representing the Euclidean distance between the closest pair of points, rounded off to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
0 3
1 1
10 18
4 9

Output: Closest pair of points: (0, 3) and (1, 1)
Distance: 2.24

Answer

```
#include <stdio.h>
#include <math.h>
#include <float.h>

typedef struct {
    double x, y;
```

```
} Point;

double getDistance(Point p1, Point p2) {
    return sqrt((p1.x - p2.x) * (p1.x - p2.x) + (p1.y - p2.y) * (p1.y - p2.y));
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    Point pts[n];
    for (int i = 0; i < n; i++) {
        scanf("%lf %lf", &pts[i].x, &pts[i].y);
    }

    double min_dist = DBL_MAX;
    Point p1_res, p2_res;

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            double d = getDistance(pts[i], pts[j]);
            if (d < min_dist) {
                min_dist = d;
                p1_res = pts[i];
                p2_res = pts[j];
            }
        }
    }

    printf("Closest pair of points: (%g, %g) and (%g, %g)\n", p1_res.x, p1_res.y,
p2_res.x, p2_res.y);
    printf("Distance: %.2f\n", min_dist);

    return 0;
}
```

Status : Correct **Marks : 10/10**

5. Problem Statement:

Lena, a geologist, is studying the boundaries of a newly discovered forest reserve. To define the outer edges of the reserve, Lena needs to identify the convex hull from a set of points marking different trees within the area.

Using Graham's Scan algorithm, help Lena compute the convex hull to outline the forest reserve's perimeter.

Example1

Input:

6

0 0

1 1

2 2

4 4

0 4

4 0

Output:

Convex Hull:

(0, 0)

(4, 0)

(4, 4)

(0, 4)

Explanation:

Pivot Selection: The bottom-most point is (0, 0).

Sorting by Polar Angle: Points sorted as (0, 0), (4, 0), (4, 4), (0, 4), (1, 1), (2, 2).

Convex Hull Construction: Start with (0, 0), and include points forming counterclockwise turns.

The final hull is: (0, 0), (4, 0), (4, 4), (0, 4).

Example2

Input:

3

0 0

0 3

3 0

Output:

Convex Hull:

(0, 0)

(3, 0)

(0, 3)

Explanation:

Pivot Selection: The bottom-most point is (0, 0).

Sorting by Polar Angle: Points sorted as (0, 0), (3, 0), (0, 3).

Convex Hull Construction: Start with (0, 0), and include points forming counterclockwise turns.

The final hull is: (0, 0), (3, 0), (0, 3).

Input Format

The first line of input contains an integer n, representing the number of trees in the forest reserve.

The next n lines contain two integers x and y, representing the coordinates of each tree.

Output Format

The output prints "Convex Hull:" followed by the coordinates of the trees forming the convex hull in counterclockwise order in new line.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6
0 0
1 1
2 2
4 4
0 4
4 0
Output: Convex Hull:
(0, 0)
(4, 0)
(4, 4)
(0, 4)

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

typedef struct {
    int x, y;
} Point;

Point pivot;

int distSq(Point p1, Point p2) {
    return (p1.x - p2.x) * (p1.x - p2.x) + (p1.y - p2.y) * (p1.y - p2.y);
}

int orientation(Point p, Point q, Point r) {
    int val = (q.y - p.y) * (r.x - q.x) - (q.x - p.x) * (r.y - q.y);
    if (val == 0) return 0;
    return (val > 0) ? 1 : 2;
}

int compare(const void* vp1, const void* vp2) {
    Point* p1 = (Point*)vp1;
```

```
    Point* p2 = (Point*)vp2;
    int o = orientation(pivot, *p1, *p2);
    if (o == 0)
        return (distSq(pivot, *p2) >= distSq(pivot, *p1)) ? -1 : 1;
    return (o == 2) ? -1 : 1;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    Point pts[n];
    for (int i = 0; i < n; i++) {
        scanf("%d %d", &pts[i].x, &pts[i].y);
    }

    if (n < 3) {
        printf("Convex Hull:\n");
        for (int i = 0; i < n; i++) printf("(%d, %d)\n", pts[i].x, pts[i].y);
        return 0;
    }

    int ymin = 0;
    for (int i = 1; i < n; i++) {
        if (pts[i].y < pts[ymin].y || (pts[i].y == pts[ymin].y && pts[i].x < pts[ymin].x))
            ymin = i;
    }

    Point temp = pts[0];
    pts[0] = pts[ymin];
    pts[ymin] = temp;

    pivot = pts[0];
    qsort(&pts[1], n - 1, sizeof(Point), compare);

    int m = 1;
    for (int i = 1; i < n; i++) {
        while (i < n - 1 && orientation(pivot, pts[i], pts[i + 1]) == 0) i++;
        pts[m++] = pts[i];
    }

    if (m < 3) {
```

```
        printf("Convex Hull:\n");
        for (int i = 0; i < m; i++) printf("(%d, %d)\n", pts[i].x, pts[i].y);
        return 0;
    }

    Point stack[n];
    int top = 0;
    stack[top++] = pts[0];
    stack[top++] = pts[1];
    stack[top++] = pts[2];

    for (int i = 3; i < m; i++) {
        while (top > 1 && orientation(stack[top - 2], stack[top - 1], pts[i]) != 2) top--;
        stack[top++] = pts[i];
    }

    printf("Convex Hull:\n");
    for (int i = 0; i < top; i++) {
        printf("(%d, %d)\n", stack[i].x, stack[i].y);
    }

    return 0;
}
```

Status : Partially correct **Marks : 8.5/10**